

TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI TP. HỒ CHÍ MINH

VIỆN ĐÀO TẠO CHẤT LƯỢNG CAO
★★★



BÀI BÁO CÁO

E-COMMERCE DATA LAKEHOUSE THIẾT KẾ VÀ TRIỂN KHAI GIẢI PHÁP DATA LAKEHOUSE CHO HỆ THỐNG THƯƠNG MẠI ĐIỆN TỬ

Môn : Big Data và ứng dụng

Mã lớp : 010412711004

Giảng viên HD : Hàn Trung Định

Nhóm : 6

Sinh viên thực hiện :

STT	Họ và tên	MSSV	Đánh giá
1	Huỳnh Thế Hy	051205009083	100%
2	Nghiêm Đức Thuận	060205003756	100%
3	Phan Trần Nhật Hưng	079205042911	100%
4	Nguyễn Khánh Dương	042205000026	100%

TP. Hồ Chí Minh, ngày 28 tháng 06 năm 2026.

LỜI CAM ĐOAN

Chúng em cam đoan đây là bài báo cáo do chính chúng em thực hiện. Các số liệu, kết quả nêu trong báo cáo là trung thực và chưa từng được công bố trong bất kỳ công trình nào khác. Mọi tài liệu tham khảo đều được trích dẫn đầy đủ và rõ ràng. Chúng em hoàn toàn chịu trách nhiệm về nội dung của bài báo cáo này.

TP. Hồ Chí Minh, tháng 06 năm 2026

Sinh viên thực hiện

MỤC LỤC

LỜI CAM ĐOAN	I
MỤC LỤC	II
DANH MỤC CÁC TỪ VIẾT TẮT	IV
MỤC LỤC HÌNH ẢNH	V
CHƯƠNG 1: TỔNG QUAN VỀ DATA LAKEHOUSE	1
1.1. Giới thiệu và bối cảnh bài toán.....	1
1.2. So sánh Data Lake, Data Warehouse và Data Lakehouse.....	1
1.2.1. Data Lake.....	1
1.2.2. Data Warehouse.....	1
1.2.3. Data Lakehouse	1
1.3. Kiến trúc Delta Lake và Apache Spark	2
1.4. Sơ đồ Diagram.....	2
CHƯƠNG 2: THIẾT KẾ HỆ THỐNG	3
2.1. Kiến trúc tổng thể (Bronze – Silver – Gold)	3
2.1.1. Bronze Layer – Ingestion	3
2.1.2. Silver Layer – Cleansing & Deduplication	4
2.1.3. Gold Layer – Business Aggregation	5
2.2. Cấu trúc dữ liệu và Schema.....	6
2.3. Chiến lược xử lý Duplicate.....	7
CHƯƠNG 3: TRIỂN KHAI VÀ KẾT QUẢ.....	8
3.1. Môi trường và công cụ sử dụng.....	8
3.2. Pipeline end-to-end.....	8
3.3. Kết quả kiểm tra và minh họa Delta Lake features	8
3.3.1. ACID Transactions – MERGE INTO	8
3.3.2. Time Travel	9

3.3.3. DESCRIBE HISTORY	9
KẾT LUẬN	10
TÀI LIỆU THAM KHẢO	11
PHỤ LỤC	12

DANH MỤC CÁC TỪ VIẾT TẮT

Từ viết tắt	Giải thích
ACID	Atomicity, Consistency, Isolation, Durability
API	Application Programming Interface
BI	Business Intelligence
CDC	Change Data Capture
CSV	Comma-Separated Values
DWH	Data Warehouse
ETL	Extract, Transform, Load
GCS	Google Cloud Storage
JSON	JavaScript Object Notation
ML	Machine Learning
S3	Amazon Simple Storage Service
SQL	Structured Query Language
VNĐ	Việt Nam Đồng

MỤC LỤC HÌNH ẢNH

Hình 1.1 Sơ đồ Diagram.....	2
Hình 2.1 Thông tin cơ bản.....	3
Hình 2.2 Mẫu 5 bản ghi.....	3
Hình 2.3 SILVER vs BRONZE — So sánh sau khi làm sạch	4
Hình 2.4 Phân phối đơn hàng	4
Hình 2.5 Xu hướng đơn hàng và doanh thu theo tháng.	5
Hình 2.6 fact_orders	5
Hình 2.7 Phân tích theo Region.....	5
Hình 2.8 Doanh thu theo Category sản phẩm	6
Hình 2.9 Phân tích High Value Orders.....	6
Hình 2.10 Phân tích Duplicate tại Bronze layer.....	7
Hình 3.1 DESCRIBE HISTORY — Silver orders.....	9
Hình 3.2 DESCRIBE HISTORY — Gold fact_orders	9

CHƯƠNG 1: TỔNG QUAN VỀ DATA LAKEHOUSE

1.1. Giới thiệu và bối cảnh bài toán

Trong bối cảnh thương mại điện tử phát triển nhanh chóng, các công ty phải xử lý khối lượng dữ liệu khổng lồ từ nhiều nguồn khác nhau như đơn hàng, sự kiện người dùng, thông tin sản phẩm và khách hàng. Bài tập này đặt ra tình huống thực tế: công ty đang vận hành song song hai hệ thống lưu trữ và cả hai đều gặp vấn đề nghiêm trọng.

Data Lake hiện tại lưu trữ dữ liệu thô trên object storage (S3) với chi phí thấp, nhưng không có schema enforcement nên dữ liệu lộn xộn, tồn tại nhiều bản ghi trùng lặp (duplicate) do lỗi double-insert từ các service. Data Warehouse (Redshift/BigQuery) tuy đảm bảo chất lượng dữ liệu nhưng chi phí vận hành rất cao, quá trình ETL chậm và khó hỗ trợ real-time. Analytics team đang cần dữ liệu đáng tin cậy và gần real-time để ra quyết định kinh doanh, nhưng hệ thống hiện tại không đáp ứng được.

1.2. So sánh Data Lake, Data Warehouse và Data Lakehouse

1.2.1. Data Lake

Data Lake là kho lưu trữ tập trung cho phép lưu trữ dữ liệu cấu trúc, bán cấu trúc và phi cấu trúc ở dạng thô. Ưu điểm chính là chi phí lưu trữ thấp (Parquet trên S3) và linh hoạt cao. Tuy nhiên, Data Lake thiếu ACID transactions, không có schema enforcement và không hỗ trợ MERGE/UPDATE native, dẫn đến vấn đề dữ liệu lộn xộn và khó kiểm soát chất lượng.

1.2.2. Data Warehouse

Data Warehouse là hệ thống lưu trữ dữ liệu đã được xử lý và tổ chức, tối ưu cho việc truy vấn phân tích. Data Warehouse đảm bảo ACID transactions, schema chặt chẽ và hiệu năng query cao. Nhưng chi phí vận hành rất đắt, khó hỗ trợ streaming real-time và mỗi lần thay đổi schema phải tái cấu trúc toàn bộ.

1.2.3. Data Lakehouse

Data Lakehouse là paradigm mới kết hợp ưu điểm của cả hai: lưu trữ rẻ như Data Lake (Parquet trên object storage) đồng thời có ACID, schema enforcement và query hiệu năng cao như Data Warehouse, nhờ lớp metadata là Delta Lake transaction log. Đây là giải pháp phù hợp nhất cho bài toán của công ty.

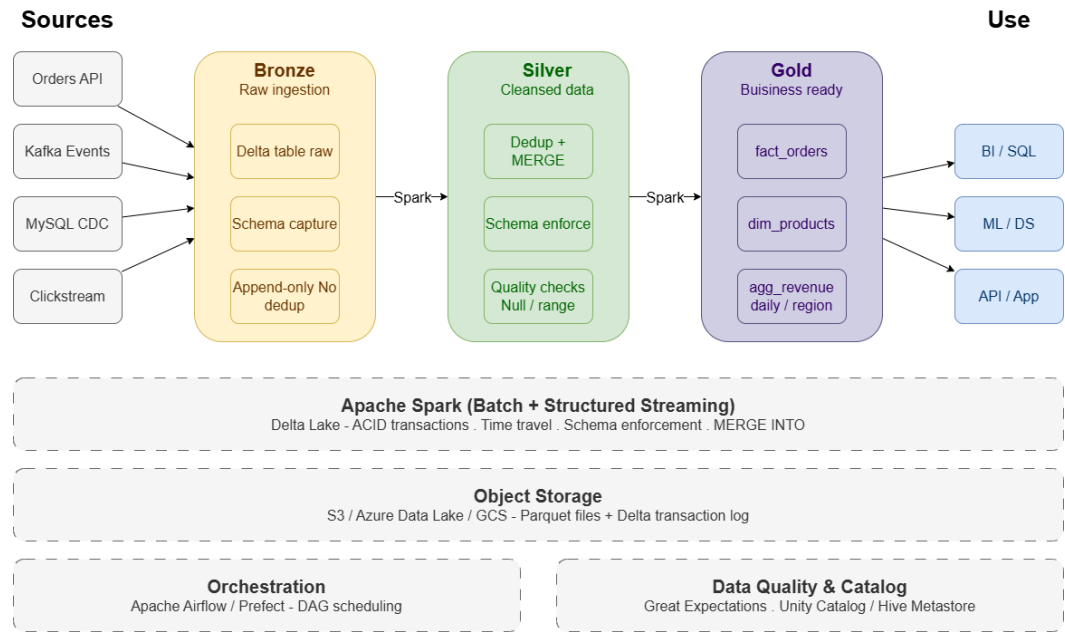
Tiêu chí	Data Lake	Data Warehouse	Lakehouse
Chi phí lưu trữ	Thấp	cao	Thấp
ACID Transactions	Không	có	Có
Schema Enforcement	không	Có	Có
Streaming Support	Phức tạp	Khó	native
Time Travel	Không	Không	Có
MERGE INTO	Không	Có	Có
ML Workloads	Không	trung bình	Tốt

Bảng 1.1: So sánh các paradigm lưu trữ dữ liệu

1.3. Kiến trúc Delta Lake và Apache Spark

Delta Lake là open-source storage layer chạy trên Parquet, bổ sung ACID transactions thông qua transaction log (_delta_log/). Apache Spark phiên bản 3.5 đảm nhận toàn bộ engine xử lý dữ liệu: từ batch ingestion đến Structured Streaming, cho phép xử lý cả batch và real-time trong cùng một framework.

1.4. Sơ đồ Diagram



Hình 1.1 Sơ đồ Diagram

CHƯƠNG 2: THIẾT KẾ HỆ THỐNG

2.1. Kiến trúc tổng thể (Bronze – Silver – Gold)

Dự án áp dụng kiến trúc Medallion Architecture gồm ba lớp xử lý tuần tự. Mỗi lớp có trách nhiệm rõ ràng và tạo ra dữ liệu ngày càng sạch hơn, đáng tin cậy hơn.

2.1.1. Bronze Layer – Ingestion

Bronze là lớp ingestion, nạp dữ liệu nguyên trạng (as-is) từ các nguồn vào Delta table theo chế độ append-only. Dữ liệu không bị làm sạch hay chuyển đổi tại đây. Mọi bản ghi trùng lặp vẫn được giữ nguyên để đảm bảo tính toàn vẹn lịch sử. Hai module chính:

- ingest_orders.py: đọc CSV đơn hàng, thêm metadata (_bronze_ingested_at, _source_file)
- ingest_events.py: Structured Streaming từ JSON files, checkpoint-based, trigger 10s

```
Số bản ghi (bao gồm duplicate) : 1,100
Số cột                          : 10

root
|-- order_id: string (nullable = true)
|-- customer_id: string (nullable = true)
|-- product_id: string (nullable = true)
|-- quantity: integer (nullable = true)
|-- unit_price: double (nullable = true)
|-- status: string (nullable = true)
|-- order_date: string (nullable = true)
|-- ingested_at: string (nullable = true)
|-- _bronze_ingested_at: string (nullable = true)
|-- _source_file: string (nullable = true)
```

Hình 2.1 Thông tin cơ bản

```
Mẫu 5 bản ghi từ Bronze:
+-----+-----+-----+-----+-----+-----+-----+-----+
|order_id|customer_id|product_id|quantity|unit_price|status|order_date|ingested_at|
+-----+-----+-----+-----+-----+-----+-----+-----+
|ORD-000196|CUST-0107|PROD-0047|2|2147000.0|delivered|2024-07-23|2026-06-26T21:40:49|
|ORD-000080|CUST-0098|PROD-0022|1|873000.0|shipped|2024-12-08|2026-06-26T21:40:49|
|ORD-000481|CUST-0098|PROD-0022|8|3383000.0|delivered|2024-11-13|2026-06-26T21:40:49|
|ORD-000110|CUST-0039|PROD-0018|3|867000.0|pending|2024-11-27|2026-06-26T21:40:49|
|ORD-000281|CUST-0062|PROD-0047|2|1231000.0|cancelled|2024-06-25|2026-06-26T21:45:49|
+-----+-----+-----+-----+-----+-----+-----+-----+
only showing top 5 rows
```

Hình 2.2 Mẫu 5 bản ghi

2.1.2. Silver Layer – Cleansing & Deduplication

Silver là lớp xử lý và làm sạch dữ liệu từ Bronze. Đây là lớp quan trọng nhất vì giải quyết trực tiếp vấn đề duplicate và schema lỏng lẻo. Chiến lược dedup sử dụng Window function ROW_NUMBER() OVER (PARTITION BY order_id ORDER BY ingested_at DESC), chỉ giữ bản ghi mới nhất cho mỗi order_id. Sau đó áp dụng MERGE INTO (ACID) để upsert vào Silver table, đảm bảo idempotency: chạy lại nhiều lần không tạo ra dữ liệu sai.

```
Bronze rows : 1,100 (raw, có duplicate)
Silver rows : 500 (sạch, unique order_id)
Đã loại bỏ : 600 bản ghi (54.5%)

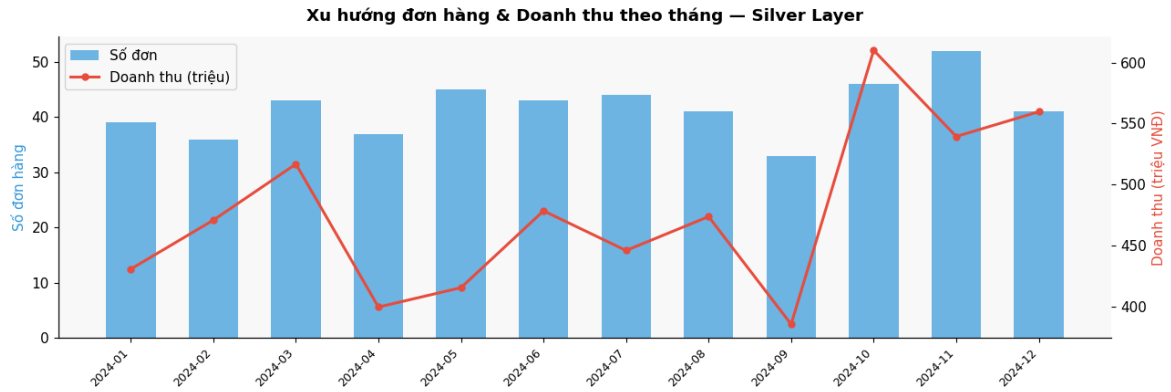
Silver schema (typed, strict hơn Bronze):
root
|-- order_id: string (nullable = true)
|-- customer_id: string (nullable = true)
|-- product_id: string (nullable = true)
|-- quantity: integer (nullable = true)
|-- unit_price: double (nullable = true)
|-- status: string (nullable = true)
|-- order_date: date (nullable = true)
|-- total_amount: double (nullable = true)
|-- _silver_updated_at: timestamp (nullable = true)
```

Hình 2.3 SILVER vs BRONZE — So sánh sau khi làm sạch

Silver Layer — Phân phối đơn hàng



Hình 2.4 Phân phối đơn hàng



Hình 2.5 Xu hướng đơn hàng và doanh thu theo tháng.

2.1.3. Gold Layer – Business Aggregation

Gold là lớp cuối cùng, tạo ra dữ liệu business-ready theo dạng Star Schema. fact_orders join ba bảng Silver orders, customers và products để tạo bảng giàu thông tin với customer_name, region, product_name, category, is_high_value. agg_revenue_daily tổng hợp doanh thu theo ngày × region × category phục vụ Analytics team.

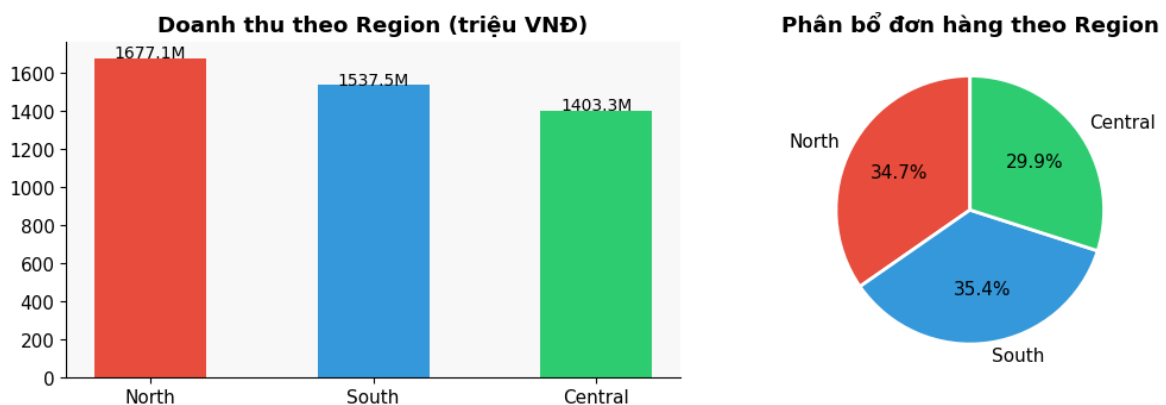
Số bản ghi : 500
Partitions : order_year × order_month

order_id	order_date	customer_name	region	product_name	category	total_amount	is_high_value	status
ORD-000005	2024-08-12	Hương Vũ	South	Tùng Beauty 46	Beauty	1.876E7	true	cancelled
ORD-000015	2024-08-08	Hà Hoàng	Central	Như Electronics 3	Electronics	2.0065E7	true	pending
ORD-000036	2024-08-26	Bác Trung Lê	North	Tùng Beauty 29	Beauty	4432000.0	true	delivered
ORD-000040	2024-08-31	Quý cô Xuân Dương	Central	Cửa Home & Living 50	Home & Living	8376000.0	true	delivered
ORD-000043	2024-08-05	Dũng Mai	North	Có Beauty 12	Beauty	3.7692E7	true	pending

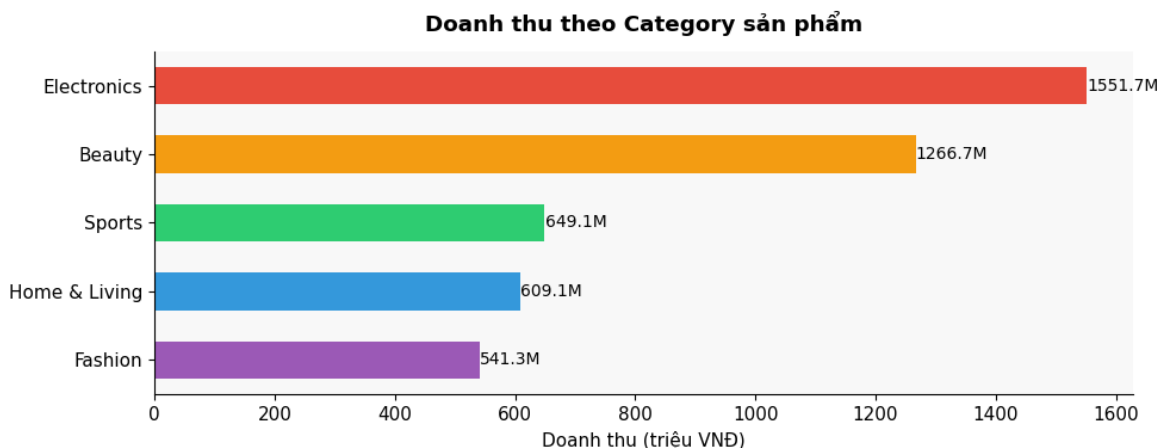
only showing top 5 rows

Hình 2.6 fact_orders

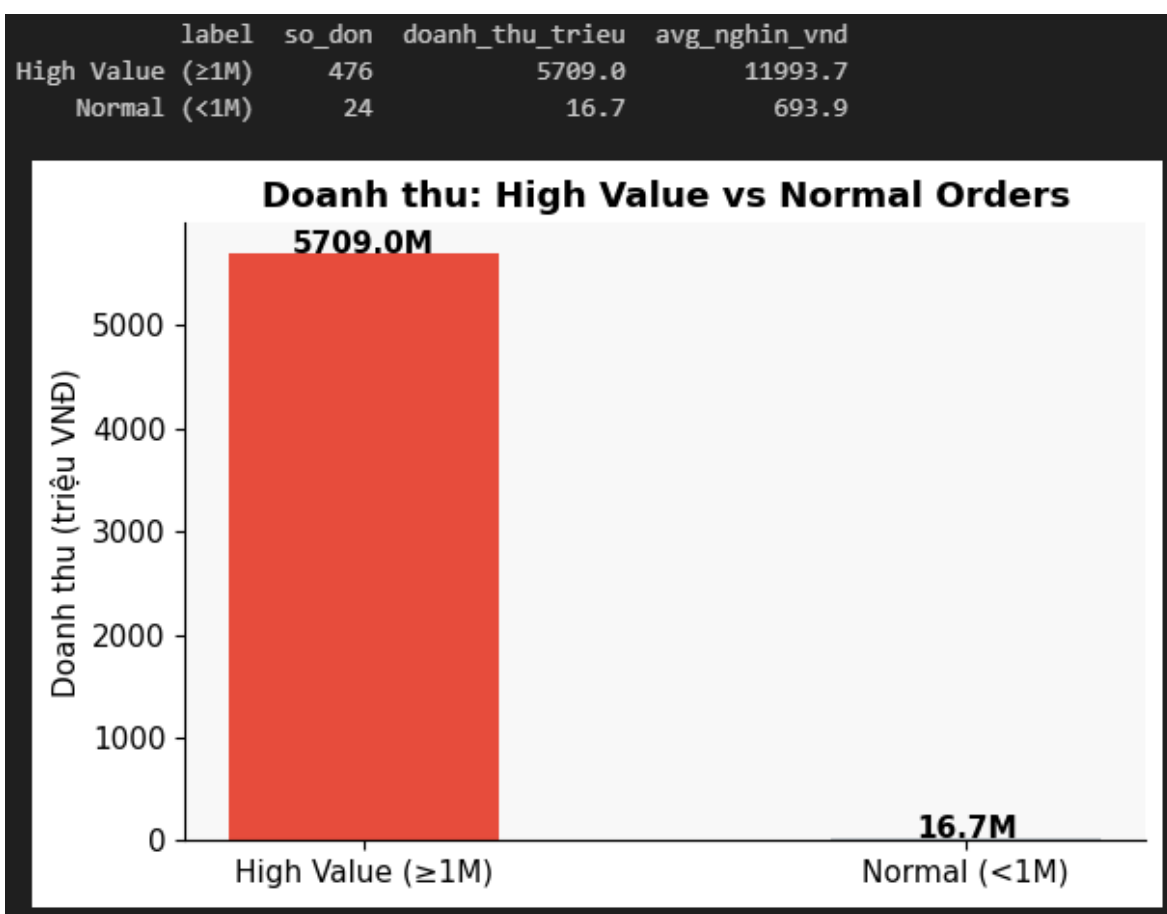
Gold Layer — Phân tích theo Region



Hình 2.7 Phân tích theo Region



Hình 2.8 Doanh thu theo Category sản phẩm



Hình 2.9 Phân tích High Value Orders

2.2. Cấu trúc dữ liệu và Schema

Mỗi layer có schema riêng với mức độ chặt chẽ tăng dần. Bronze dùng string cho hầu hết các cột để tránh lỗi đọc. Silver ép kiểu chính xác: order_date sang DateType, quantity sang IntegerType, unit_price sang DoubleType, đồng thời thêm

cột tính toán $\text{total_amount} = \text{quantity} \times \text{unit_price}$. Gold partition theo `order_year` và `order_month` để tối ưu query theo thời gian nhờ partition pruning.

2.3. Chiến lược xử lý Duplicate

Vấn đề duplicate được mô phỏng trong `generate_data.py` với tỷ lệ 10%: 500 đơn hàng gốc cộng thêm ~50 bản ghi duplicate có `ingested_at` khác nhau (cách 5 phút). Lý do không dùng `dropDuplicates()` đơn giản: vì các bản ghi duplicate có `ingested_at` khác nhau nên `dropDuplicates` trên toàn cột sẽ không phát hiện được. Window function với `PARTITION BY order_id` và sắp xếp theo `ingested_at DESC` đảm bảo luôn giữ bản ghi mới nhất, bất kể số lần duplicate.

```
Tổng bản ghi      : 1,100
Unique order_id   : 500
Số duplicate      : 600
Tỷ lệ duplicate   : 54.5%
```

Hình 2.10 Phân tích Duplicate tại Bronze layer

CHƯƠNG 3: TRIỂN KHAI VÀ KẾT QUẢ

3.1. Môi trường và công cụ sử dụng

Công cụ / Thư viện	Phiên bản & Mục đích
Python	3.10+ — ngôn ngữ chính
Apache Spark	3.5.8 — engine xử lý batch và streaming
Delta Lake (delta-spark)	3.0.0 — ACID, Time Travel, MERGE INTO
Faker	24.0.0 — sinh dữ liệu mẫu giả lập
Pandas	2.1.4 — đọc/ghi CSV trong generate_data.py
Java	11 — runtime bắt buộc cho PySpark

Bảng 3.1: Môi trường và công cụ triển khai

3.2. Pipeline end-to-end

Pipeline được thiết kế chạy end-to-end với hai lệnh đơn giản:

```
python generate_data.py # Sinh 550 bản ghi (500 + 50 duplicate)
```

```
python pipeline.py      # Chạy toàn bộ Bronze → Silver → Gold
```

pipeline.py điều phối tuần tự 6 bước: kiểm tra đầu vào → Bronze ingest orders → Silver clean orders (MERGE INTO) → Silver clean customers → Gold fact_orders (JOIN) → Gold agg_revenue_daily (GROUP BY) → in báo cáo tóm tắt và Time Travel info. Kết quả mong đợi: Bronze 550 bản ghi → Silver 500 bản ghi (loại 50 duplicate) → Gold fact 500 bản ghi.

3.3. Kết quả kiểm tra và minh họa Delta Lake features

3.3.1. ACID Transactions – MERGE INTO

MERGE INTO là tính năng quan trọng nhất được minh họa trong dự án. Trong silver_orders.py, MERGE INTO với điều kiện target.order_id = source.order_id đảm bảo: nếu order đã tồn tại thì UPDATE, nếu chưa có thì INSERT. Tính idempotency được kiểm chứng: chạy pipeline nhiều lần liên tiếp không tạo ra bản ghi trùng lặp. Delta Lake đảm bảo tính atomicity: nếu MERGE bị interrupt giữa chừng, dữ liệu sẽ rollback về trạng thái trước đó.

3.3.2. Time Travel

Sau mỗi lần chạy pipeline, Delta Lake tạo ra một version mới trong `_delta_log/`. `demo.ipynb` minh họa đầy đủ: đọc lại dữ liệu tại version 0 bằng `versionAsOf=0`, đọc theo timestamp bằng `timestampAsOf`, và demo live update một bản ghi rồi so sánh nội dung trước và sau bằng version number. Đây là tính năng đặc biệt của Lakehouse mà cả Data Lake lẫn Data Warehouse truyền thống đều không có.

3.3.3. DESCRIBE HISTORY

Transaction log của Delta Lake ghi lại mọi thao tác ghi với đầy đủ metadata: version, timestamp, operation (WRITE/MERGE/UPDATE), và operationMetrics chi tiết số bản ghi inserted/updated/deleted. Trong `demo.ipynb`, `DeltaTable.forPath(spark, path).history()` trả về toàn bộ lịch sử, tương đương lệnh SQL: `DESCRIBE HISTORY delta.`path``. Đây là cơ sở của tính năng Audit Trail trong môi trường production.

version	timestamp	operation	operationMetrics	userName
1	2026-06-26 21:56:19.825	MERGE	{numTargetRowsCopied -> 0, numTargetRowsDeleted -> 0, num...	NULL
0	2026-06-26 21:41:46.058	WRITE	{numFiles -> 1, numOutputRows -> 500, numOutputBytes -> 1...}	NULL

version	timestamp	operation
1	2026-06-26 21:56:19.825	MERGE
0	2026-06-26 21:41:46.058	WRITE

Hình 3.1 DESCRIBE HISTORY — Silver orders

version	timestamp	operation	operationMetrics
1	2026-06-26 21:56:28.541	WRITE	{numFiles -> 12, numOutputRows -> 500, numOutputBytes -> 82063}
0	2026-06-26 21:41:54.632	WRITE	{numFiles -> 12, numOutputRows -> 500, numOutputBytes -> 82063}

Hình 3.2 DESCRIBE HISTORY — Gold fact_orders

KẾT LUẬN

Bài báo cáo đã trình bày toàn bộ quá trình thiết kế và triển khai một giải pháp Data Lakehouse hoàn chỉnh cho bài toán thương mại điện tử. Qua quá trình thực hiện, các mục tiêu học thuật đã được đạt được:

Thứ nhất, đã hiểu rõ và thực hành được kiến trúc Lakehouse paradigm thông qua việc xây dựng pipeline Bronze → Silver → Gold, giải quyết trực tiếp ba vấn đề đặt ra trong đề bài: data lake lộn xộn, data warehouse đắt tiền và nhu cầu real-time.

Thứ hai, đã thực hành thành thạo ACID tables với Delta Lake qua MERGE INTO, schema enforcement ngăn chặn schema drift, và versioning qua Time Travel. Đặc biệt chiến lược dedup bằng Window function cho thấy sự hiểu biết sâu về xử lý dữ liệu thực tế.

Thứ ba, đã xây dựng được pipeline end-to-end hoàn chỉnh sử dụng Apache Spark với cả batch processing và Structured Streaming, minh họa khả năng xử lý real-time của kiến trúc Lakehouse.

Thứ tư, đã thực hiện so sánh chi tiết ba paradigm Data Lake, Data Warehouse và Data Lakehouse, đưa ra lý luận rõ ràng tại sao Lakehouse là lựa chọn phù hợp nhất cho bài toán của công ty thương mại điện tử.

Sản phẩm bàn giao bao gồm: sơ đồ kiến trúc (.drawio), mã nguồn đầy đủ 11 files Python và notebook demo.ipynb minh họa EDA, Time Travel và DESCRIBE HISTORY. Toàn bộ project có thể chạy ngay với hai lệnh đơn giản.

TÀI LIỆU THAM KHẢO

- [1] Armbrust, M., et al. (2020). Delta Lake: High-Performance ACID Table Storage over Cloud Object Stores. *Proceedings of the VLDB Endowment*, 13(12), 3411–3424.
- [2] Databricks Inc. (2023). Delta Lake Documentation, 11/2023, <https://docs.delta.io/latest/index.html>
- [3] Apache Software Foundation. (2024). Apache Spark Documentation – Structured Streaming, 03/2024, <https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html>
- [4] Garg, S. (2022). *Data Lakehouse Architecture and AI*. O'Reilly Media, Sebastopol, CA.
- [5] Kimball, R., & Ross, M. (2013). *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling* (3rd ed.). Wiley, Indianapolis, IN.
- [6] Zaharia, M., et al. (2016). Apache Spark: A Unified Engine for Big Data Processing. *Communications of the ACM*, 59(11), 56–65.

PHỤ LỤC

Phụ lục A: Cấu trúc thư mục dự án

```
ecommerce-lakehouse/  
├── README.md  
├── requirements.txt  
├── generate_data.py  
├── pipeline.py  
├── config/  
│   └── spark_session.py  
├── bronze/  
│   ├── ingest_orders.py  
│   └── ingest_events.py  
├── silver/  
│   ├── silver_orders.py  
│   └── silver_customers.py  
├── gold/  
│   ├── gold_fact_orders.py  
│   └── gold_agg_revenue.py  
├── notebooks/  
│   └── demo.ipynb  
└── diagrams/  
    └── lakehouse_architecture.drawio
```

Phụ lục B: Lệnh chạy dự án

```
# Bước 1: Cài đặt môi trường  
python -m venv .venv && source .venv/bin/activate  
pip install -r requirements.txt
```

Bước 2: Sinh dữ liệu mẫu

python generate_data.py

Bước 3: Chạy pipeline

python pipeline.py

Bước 4: Mở notebook demo

jupyter notebook notebooks/demo.ipynb

Link minh chứng vẽ diagram:

<https://drive.google.com/file/d/1YgKnuUvde929yNlRL7DaLxA3bYIkGS8k/view?usp=sharing>